



AI 서비스를 위한 SW 기초 Full-stack Engineering

스마트 데이터 이해 및 활용

Day03_종합실습_평가용

2026.07

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SQL 튜닝 종합실습

50,000명 HR 데이터로 익히는 실행 계획 기반 문제 해결

제공 : PostgreSQL_day03_tuning_샘플_스키마_DDL_DML.sql

- schema : day03_tuning
- datas :
 - ✓ department_count : 20 rows
 - ✓ employee_count : 50,000 rows
 - ✓ employee_work_log_count : 500,000 rows
 - ✓ employee_trading_count : 45,001 rows

LAB SCOPE

10문항

왜 이 실습인가

상황

사원 수가 5만 명을 넘으면서
검색 응답과 보고서 생성이
눈에 띄게 느려졌다.

50,000

employees

500,000

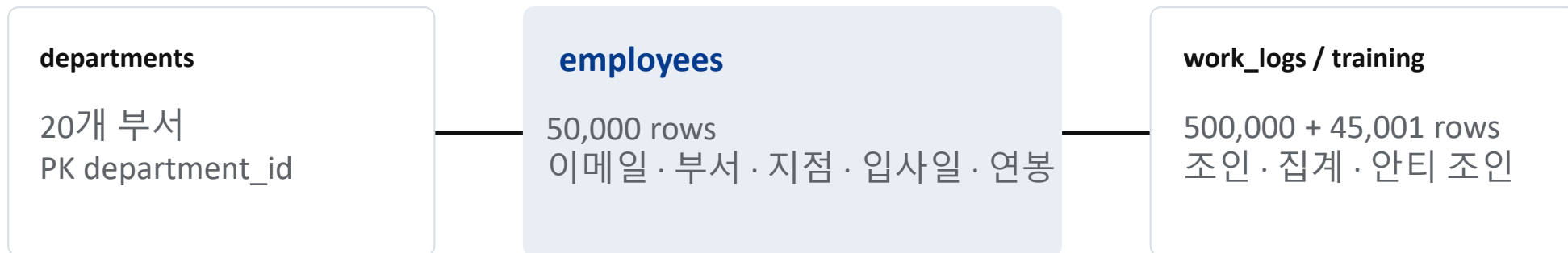
employee_work_logs

제출 자료

- 01 느린 이유와 실행 계획 근거
- 02 개선 방법과 설계 이유
- 03 개선 전·후 실행 계획 캡처
- 04 결과 정확성 검증

비교 기준 Execution Time · Buffers · actual/estimated rows · loops

데이터 모델과 공통 실습 루프



모든 문제의 공통 루프

01 BASELINE
실행 계획·수치 저장

02 HYPOTHESIS
병목 원인 가설

03 CHANGE
SQL·인덱스 변경

04 REMEASURE
수치·정확성 재검증

PROBLEM 01 · PLAN READING

기본 키 단건 조회 — 문제 1. 기본 키 검색의 실행 계획 읽기

느린 SQL

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, TIMING OFF)
SELECT *
FROM day03_tuning.employees
WHERE employee_id = 100;
```

수행 과제

1. 사용된 Scan 노드와 Index Cond를 확인한다.
2. estimated rows와 actual rows를 비교한다.
3. Buffers와 Execution Time을 기록한다.
4. 추가 인덱스가 필요한지 근거로 판단한다.

목표: 빠르게 만드는 것보다 '튜닝 불필요'를 증명하기

PROBLEM 02 · EXPRESSION

대소문자 무시 이메일 검색 — 함수가 인덱스를 가린다

느린 SQL

```
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF)
SELECT employee_id, employee_no, full_name, email
FROM day03_tuning.employees
WHERE lower(email) = 'user1234@corp.com';
```

수행 과제

1. 개선 전 Rows Removed by Filter를 확인한다.
2. WHERE절 표현식과 일치하는 인덱스를 설계한다.
3. 개선 전·후 Scan 노드와 Buffers를 비교한다.
4. 저장값 소문자 정규화 대안도 설명한다.

관찰: 일반 email 인덱스와 lower(email)은 같은 검색식인가?

PROBLEM 03 · LIKE · TRIGRAM

접미사 LIKE 검색 — 선행 와일드카드를 어떻게 다룰까?

느린 SQL

```
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF)
SELECT employee_id, employee_no, full_name, email
FROM day03_tuning.employees
WHERE email LIKE '%@gmail.com';
```

수행 과제

1. 일반 B-tree가 어려운 이유를 설명한다.
2. pg_trgm 기반 인덱스 후보를 선택한다.
3. 개선 후 Bitmap 관련 노드를 확인한다.
4. 결과 비율이 높을 때 계획이 달라질 이유를 쓴다.

선택도: gmail 계정은 전체의 약 1%

PROBLEM 04 · FILTER · ORDER · LIMIT

재직자 최근 입사자 — 연봉 상위 100명을 빠르게

느린 SQL

```
SELECT employee_id, employee_no, full_name,  
       hire_date, salary  
FROM day03_tuning.employees  
WHERE employment_status = 'ACTIVE'  
      AND hire_date >= current_date - 365  
ORDER BY salary DESC  
LIMIT 100;
```

수행 과제

1. 개선 전 Sort 노드와 Sort Method를 확인한다.
2. 필터와 정렬 중 어떤 순서를 인덱스에 반영할지 결정한다.
3. ACTIVE 행만 대상으로 하는 설계를 검토한다.
4. LIMIT 조기 종료가 실제로 일어났는지 확인한다.

정답은 하나가 아니다: 데이터 선택도와 읽은 행 수로 판단

PROBLEM 05 · OR · IN

세 지점 검색 — OR과 IN은 정말 성능이 다를까?

느린 SQL

```
SELECT employee_id, employee_no,  
       full_name, branch_code  
FROM day03_tuning.employees  
WHERE branch_code = 'B003'  
       OR branch_code = 'B004'  
       OR branch_code = 'B005';
```

수행 과제

1. 인덱스가 없을 때의 기준 계획을 저장한다.
2. branch_code 인덱스를 설계한다.
3. 같은 조건을 IN으로 다시 작성한다.
4. OR/IN 계획·시간·가독성을 각각 평가한다.

표현을 바꾸는 것과 접근 경로를 만드는 것은 다른 문제

PROBLEM 06 · SARGABILITY

2025년 입사자 — 컬럼에 함수가 적용된 날짜 조건

느린 SQL

```
SELECT employee_id, employee_no,  
       full_name, hire_date  
FROM day03_tuning.employees  
WHERE extract(year FROM hire_date) = 2025;
```

수행 과제

1. hire_date 단일 인덱스를 만든다.
2. 함수 조건에서 Index Cond가 생기는지 확인한다.
3. 2025년을 반개구간 날짜 범위로 재작성한다.
4. 두 계획의 Buffers와 필터 제거 행을 비교한다.

힌트: 2025-01-01 이상, 2026-01-01 미만

PROBLEM 07 · COMPOSITE INDEX

부서 + 직무 + 연봉순 — 복합 인덱스의 컬럼 순서

느린 SQL

```
SELECT employee_id, employee_no,  
       full_name, salary  
FROM day03_tuning.employees  
WHERE department_id = 5  
       AND job_code = 'DEV'  
ORDER BY salary DESC  
LIMIT 20;
```

수행 과제

1. 등호 조건과 정렬 컬럼의 순서를 설계한다.
2. 개선 후 별도 Sort가 제거되는지 확인한다.
3. department_id 없이 job_code만 검색한다.
4. 왼쪽 선두 컬럼의 의미를 실행 계획으로 설명한다.

인덱스 순서는 WHERE절에 적힌 순서와 자동으로 같지 않다

PROBLEM 08 · COVERING INDEX

사번 구간 조회 — 출력 컬럼까지 인덱스가 담당할 수 있을까?

느린 SQL

```
SELECT employee_no, full_name, email
FROM day03_tuning.employees
WHERE employee_no >= 'EMP040000'
      AND employee_no < 'EMP040051'
ORDER BY employee_no;
```

수행 과제

1. 검색 키와 출력 전용 컬럼을 구분한다.
2. INCLUDE를 사용한 커버링 인덱스를 설계한다.
3. VACUUM ANALYZE 후 계획을 다시 측정한다.
4. Index Only Scan과 Heap Fetches를 함께 판독한다.

Index Only Scan이라는 이름만 보고 성공으로 판단하지 말 것

PROBLEM 09 · JOIN · LOOPS

5만 사원 × 50만 근무기록 — 최근 초과근무 집계

느린 SQL

```
SELECT e.employee_id, e.employee_no, e.full_name,  
       sum(w.overtime_minutes) AS total_overtime  
FROM day03_tuning.employees e  
JOIN day03_tuning.employee_work_logs w  
  ON w.employee_id = e.employee_id  
WHERE e.department_id = 5  
      AND e.employment_status = 'ACTIVE'  
      AND w.work_date >= current_date - 30  
GROUP BY e.employee_id, e.employee_no, e.full_name  
ORDER BY total_overtime DESC  
LIMIT 20;
```

수행 과제

1. 조인 순서와 각 노드의 실제 행 수를 확인한다.
2. 반복되는 내부 노드의 loops를 기록한다.
3. employee_id와 work_date의 복합 순서를 설계한다.
4. Hash Join/Loop 이름보다 총 Buffers와 시간을 비교한다.

loops가 큰 내부 노드는 작은 1회 비용도 크게 누적된다

PROBLEM 10 · ANTI JOIN · NULL

교육 미이수자 검색 — NOT IN 결과가 0건인 이유

느린 SQL

```
SELECT e.employee_id, e.employee_no, e.full_name
FROM day03_tuning.employees e
WHERE e.employee_id NOT IN (
  SELECT t.employee_id
  FROM day03_tuning.employee_training t
  WHERE t.completion_status = 'COMPLETED'
);
```

수행 과제

1. 서브쿼리에 NULL이 있는지 확인한다.
2. NOT IN의 3값 논리를 설명한다.
3. 정확한 안티 조인 형태로 다시 작성한다.
4. 완료 행만 지원하는 인덱스를 설계하고 재측정한다.

성능보다 먼저 결과 정확성을 회복해야 한다

CHECKLIST

제출 전 확인

[최종 제출 내용]

- | | |
|---------------------|--------------------|
| - -- 개선전 QUERY PLAN | << 복붙 |
| - -- 개선안 | <<< 이부분은 직접 작성합니다. |
| - -- 개선후 QUERY PLAN | << 복붙 |
| - -- [개선 결과 해석] | <<< 이부분은 직접 작성합니다. |
| - -- [인덱스 정의 확인] | << 복붙 |

[최종 제출 파일]

- day03_쿼리_홍길동.sql

평가 기준

총10문제 - 문항당 10점 입니다.

각 문항당 배점 :

- 개선전 QUERY PLAN : 1점
- 개선안 : 3점
- 개선후 QUERY PLAN : 1점
- [개선 결과 해석] : 4점
- [인덱스 정의 확인] : 1점

총점 : 100점

제출방법 :

- Slack > 다이렉트 메시지



수고하셨습니다.

본 문서는 SK㈜ AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.